Regular Expression (Regex) & grep

NCSC101

Prof. Pranay Kumar Saha

August 18, 2025

What is Regular Expression (Regex)?

Definition

A **Regular Expression** is a sequence of characters that defines a search pattern, mainly for string matching and manipulation.

Two perspectives:

- **Theory:** Describes *regular languages*, recognized by finite automata.
- **Practical:** A smart filter for lines, fields, filenames, and logs.

Regular Expressions in Unix Commands: RegEx

Regular expressions (REs) are concise patterns for matching text. In Unix, they are most commonly used with commands like **grep**, **sed**, and **awk**. In Vim, you can use in substitution and search.

Core Metacharacters with Examples

- **.** —matches any single character (except newline).
  Example: **c.t** matches "cat", "cot", "c9t", etc.

Regular Expressions in Unix Commands: RegEx

Core Metacharacters with Examples

- **^**, **$** —anchors start and end of line.
  Examples:
  ^Hello matches "Hello" at line start.
  end$ matches "friend" or "the end" at line end.
- *****, **+**, **?** —quantifiers:
  ***** = zero-or-more, **+** = one-or-more, **?** = zero-or-one of preceding atom.
  Examples:
  go*d matches "gd" , "god" , "goood" , ⋯
  ab+ matches "ab" , "abb" , "abbb" , ⋯
  colou?r matches "color" or "colour" .

Regular Expressions in Unix Commands: RegEx

Core Metacharacters with Examples

- **{m,n}** —match at least *m*, at most *n* occurrences of preceding atom.
  Example:
  [0-9]{2,4} matches "12", "123", or "1234" but not "1" or "12345".
- **[abc]**, **[a-z0-9]** —character classes: match exactly one of the listed characters or ranges.
  Examples:
  gr[ae]y matches "gray" or "grey".
  [A-Za-z0-9] matches any single alphanumeric.
- **[^...]** —negated class: match any character _not_ listed.
  Example: **[^0-9]** matches any non-digit character.

Regular Expressions in Unix Commands: RegEx

Core Metacharacters with Examples

- **[0-9]{10}** —Matches exactly ten consecutive ASCII digits (characters 0–9)
- **(···)** —grouping: treat the subpattern as a unit.
  Example: **(cat|dog)s?** matches "cat", "cats", "dog", or "dogs".
- **|** —alternation (logical OR, in **grep -E**).
  Example: **red|blue|green** matches any of "red", "blue", or "green".
- **** —escape the next character to match it literally.
  Example: **\.** matches a literal dot "." instead of "any character".

grep

grep (Global Regular Expression Print) searches text for patterns and outputs matching lines. Common options:

- **-i**: Case-insensitive search.
- **-v**: Invert match (select non-matching lines).
- **-c**: Count matching lines.
- **-n**: Prefix each line with its line number.
- **-w**: Match whole words only.
- **-r**: Recursive search through directories.
- **-E**: Use Extended Regular Expressions.
- **-o**: Show only the matching part of the line.

Sample Data (Create Once)

```
# Create a small sample file for all demos
cat > sample.txt << 'EOF'
Error: Disk FULL
error: network timeout
INFO: job started
Warning: low memory
info: Job finished
EOF
```

Use this **sample.txt** for the next slides.

-i —Case-insensitive match

sample.txt

```
Error: Disk FULL
error: network timeout
INFO: job started
Warning: low memory
info: Job finished
```

Command & Output

```
grep -i 'error' sample.txt
```

```
# Output:
Error: Disk FULL
error: network timeout
```

-v —Invert the match

sample.txt

```
Error: Disk FULL
error: network timeout
INFO: job started
Warning: low memory
info: Job finished
```

Command & Output

```
grep -iv '^info' sample.txt
```

```
# Output:
Error: Disk FULL
error: network timeout
Warning: low memory
```

-c —Count matching lines

sample.txt

```
Error: Disk FULL
error: network timeout
INFO: job started
Warning: low memory
info: Job finished
```

Command & Output

```
grep -ic 'job' sample.txt
```

```
# Output:
2
```

-n —Prefix line numbers

sample.txt

```
Error: Disk FULL
error: network timeout
INFO: job started
Warning: low memory
info: Job finished
```

Command & Output

```
grep -in '^warn' sample.txt
```

```
# Output:
4:Warning: low memory
```

-w —Match whole words

sample.txt

```
Error: Disk FULL
error: network timeout
INFO: job started
Warning: low memory
info: Job finished
notainfo-tag
```

Command & Output

```
grep -nw 'info' sample.txt
```

```
# Output:
3:INFO: job started
5:info: Job finished
```

-E —Extended Regular Expressions

sample.txt

```
Error: Disk FULL
error: network timeout
INFO: job started
Warning: low memory
info: Job finished
```

Command & Output

```
grep -Ein '^(Error|Warning):' sample.txt
```

```
# Output:
1:Error: Disk FULL
4:Warning: low memory
```

-o —Show only the match

sample.txt

```
Error: Disk FULL
error: network timeout
INFO: job started
Warning: low memory
info: Job finished
```

Command & Output

```
grep -Eo '^(Error|error|INFO|info|Warning)' sample.txt
```

```
# Output:
Error
error
INFO
Warning
info
```

-r —Recursive search

What it does: Search through all subdirectories and files.

```
# Search for 'timeout' in all files under logs/
grep -r 'timeout' logs/

# Limit by filename pattern (*.log)
grep -r --include='*.log' 'timeout' logs/

# Exclude a directory
grep -r --exclude-dir='sys' 'timeout' logs/
```

Challenge 1 —Lines starting with Error or Warning

sample.txt

```
Error: Disk FULL
error: network timeout
INFO: job started
Warning: low memory
info: Job finished
```

Command —What will it print?

```
grep -En '^(Error|Warning):' sample.txt
```

Expected output

```
1:Error: Disk FULL
4:Warning: low memory
```

Challenge 2 —Whole word vs substring

sample.txt

```
Error: Disk FULL
error: network timeout
INFO: job started
Warning: low memory
info: Job finished
notainfo-tag
```

Command —Predict both results

```
grep -in 'info' sample.txt
grep -inw 'info' sample.txt
```

Expected outputs

```
# substring ('info' anywhere)
3:INFO: job started
5:info: Job finished
6:notainfo-tag

# whole word only
3:INFO: job started
5:info: Job finished
```

Challenge 3 —Lines count vs occurrences

tokens.txt

```
alpha beta alpha
gamma ALPHA Beta
beta-beta
```

Command —What numbers do you expect?

```
# A) matching lines (case-insensitive)
grep -ic 'alpha' tokens.txt

# B) total occurrences using -o
grep -io 'alpha' tokens.txt | wc -l
```

Expected

```
# A
2
# B
3
```

Challenge 4 —Class ranges

ids.txt

```
CS24-001
EE23-999
ME2A-010
cs24-123
```

Command —Which lines match?

```
grep -En '^[A-Z]{2}[0-9]{2}-[0-9]{3}$' ids.txt
```

Expected

```
1:CS24-001
2:EE23-999
```

Challenge 5 —Extract severities only

logs.txt

```
INFO: start
warning: low battery
ERROR: halted
debug: trace off
Info: resumed
```

Command —What tokens will print?

```
grep -Eoi '^(error|warning|info)' logs.txt
```

Expected

```
INFO
warning
ERROR
Info
```

Note: **-o** prints only the match; line 4 is ignored.

Challenge 6 —Validate simple phone format

phones.txt

```
9876543210
+91-98765-43210
09876543210
98765 43210
```

Command —Which lines match as ”10 digits starting 6–9”?

```
grep -En '^[6-9][0-9]{9}$' phones.txt
```

Expected

```
1:9876543210
```

The --include Flag

- Restricts search to files matching a pattern
- Example: **--include='*.log'** → only log files
- Without it, grep searches all files (source code, binaries, etc.)
- Useful for focusing search on relevant files

Challenge 7 —Recursive search with filters

Tree (for illustration)

```
project/
├── app/
│   ├── today.log (contains "timeout")
│   └── old.txt
└── sys/
    └── kern.log (contains "timeout")
```

Commands —Predict outputs

```
# A) Any file
grep -rni 'timeout' project/

# B) Only *.log files
grep -rni --include='*.log' 'timeout' project/

# C) Exclude sys/ directory
grep -rni --exclude-dir='sys' 'timeout' project/
```

Expected (shape)

```
# A project/app/today.log:...timeout...
# project/sys/kern.log:...timeout...
# B project/app/today.log:...
# project/sys/kern.log:...
# C project/app/today.log:...
```

Glob Patterns (Shell Wildcards)

- '*' → zero or more characters (e.g. ls *.txt)
- '?' → exactly one character (e.g. ls ?.txt → matches a.txt, b.txt, but not ab.txt)
- '[abc]' → one of a, b, or c (e.g. ls *[ch] → matches filec, fileh.)
- '[!0-9]' → one character not a digit ls [!0-9]* → matches files not starting with a digit.
- '{a, b, c}' → brace expansion (e.g. ls *.{txt,log} → matches .txt and .log files.)
- '**' → recursive (with **shopt -s globstar**) e.g. ls **/*.txt → finds all .txt under current dir, recursively.

Glob vs Regex

Glob (Shell)

- Simple, filename expansion
- **`*.txt`** → files ending in .txt
- **`[A-Z]*.sh`** → files starting with uppercase, ending .sh

Regex (grep/sed/awk)

- Powerful, works on text contents
- **`.*\.txt$`** → lines ending with .txt
- **`^[A-Z].*\.sh$`** → lines starting with uppercase, ending .sh

Finding Files: find and locate

Two different tools for finding files by name.

find

- Searches the filesystem in real-time.
- Powerful and flexible.
- Can be slower.

```
# Find files by name
$ find /home -name "*.pdf"

# Find directories
$ find . -type d -name "images"
```

locate

- Searches a pre-built database.
- Extremely fast.
- May not find recently created files.

```
# locate is simple
$ locate my_report.docx

# Admin can update db
$ sudo updatedb
```

Counting and Sorting: sort, uniq, wc

These three utilities are often used together to process lists of text.

wc - Word Count

Counts lines, words, and characters.

```
$ wc -l user_list.txt   # Count only lines
```

Flag	Description
-l	Count lines (newline characters).
-w	Count words (whitespace□separated).
-c	Count bytes.
-m	Count characters (in multibyte locales).
-L	Print length of the longest line.

Regular Expression (Regex) & **grep**

Thank You for Listening!